REPORT

PENETRATION TESTING REPORT

DOGQUEST 2026

Date: 19-03-2026

Version: 1.0

Hachani Labiodh
Cybersecurity Specialist
213 541 623 392
staphysec@gmail.com
https://www.upwork.com/freelancers/~0133d5739153f41356

TABLE OF CONTENTS

LISTE OF TABLES

LISTE OF FIGURES

1 DOCUMENT MANAGEMENT

1.1 DOCUMENT IDENTIFICATION

Document Name	Version	Classification
DogQuest-Penetration-Test-03-2026-v1.0.pdf	1	Confidential

Table 1 - Document Identification

1.2 DOCUMENT HISTORY

Version	Date	Edits / Remarques
1.0	19-03-2026	First Version

Table 2 - Document History

1.3 DOCUMENT PREPARATION

Action	Name	Title/Role	Date
Prepared by	Hachani Labiodh	Penetration Tester	19-03-2026

Table 3 - Document Preparation

1.4 DOCUMENT APPROVAL

Name	Date	Version	Organization
		1	DogQuest

Table 4 - Document Approval

1.5 CONTRIBUTIONS (C) / DISTRIBUTIONS (D) LIST

Name	C & D	Organization	Title/Role
Hachani Labiodh	C & D	Upwork	Penetration Tester
Vian Reynecke	D	DogQuest	Project Manager

Table 5 - Contributions (C) / Distributions (D) List

2 EXECUTIVE SUMMARY

On 14-03-2026, Hachani Labiodh industry certified security professional holding the Offensive Security Certified Professional (OSCP) and Offensive Security Web Expert (OSWE) credentials (https://www.credential.net/profile/hachanilabiodh455280/wallet) conducted penetration testing for DogQuest to assess the security posture of their web application through a Grey-Box penetration test.

This report presents the identified vulnerabilities and their recommended remediations.

The assessment was conducted over 6 days, and followed best practices defined by the Open Web Application Security Project (OWASP).

2.1 SUMMARY OF STRENGHTS

While Hachani Labiodh was tasked with identifying issues and vulnerabilities within DogQuestt current environment, it is equally important to recognize and document positive findings. Highlighting the strengths of the current environment can help reinforce security best practices and provide strategic direction towards maintaining and enhancing a robust defensive posture.

A notable strength observed during this assessment was the general currency of software versions utilized across the client sites. The systems appeared to be maintained with relatively recent and supported software releases, which mitigates risks associated with known vulnerabilities present in older versions. This proactive approach to software updates demonstrates a commitment to maintaining a baseline level of security.

2.2 SUMMARY OF WEAKNESSES

During the penetration test for DogQuest, the penetration tester discovered a list of weaknesses. These weaknesses have been grouped into general categories across the current environment. The following summary highlights these weaknesses and offers recommendations for remediation to enhance the overall security posture of the enterprise:

- **Broken Access Control (Insecure Direct Object Reference):** The Supabase REST API lacks sufficient server-side authorization checks. By manipulating request headers and parameters, authenticated users can access unauthorized records.
- **Permissive Cross-Origin Resource Sharing (CORS) Policy:** The application reflects the Origin header from incoming requests directly into the Access-Control-Allow-Origin response header. This misconfiguration allows malicious websites to perform cross-origin requests and potentially exfiltrate sensitive data from authenticated users' sessions.
- **Authentication Method Enumeration:** The system's authentication endpoint reveals whether specific accounts exist or which login methods are tied to them. This information can be leveraged by attackers to conduct targeted brute-force attacks or credential stuffing against known user accounts.
- **Improper Privilege Management (Privilege Escalation):** The registration process fails to validate user-supplied roles on the server side. By intercepting and modifying request parameters during the sign-up flow, a standard user can self-assign elevated "Admin" privileges, leading to full system compromise.

Critical	High	Medium	Low	Informational	Total
1	1	2	0	0	4

Table 6 - Summary of Results

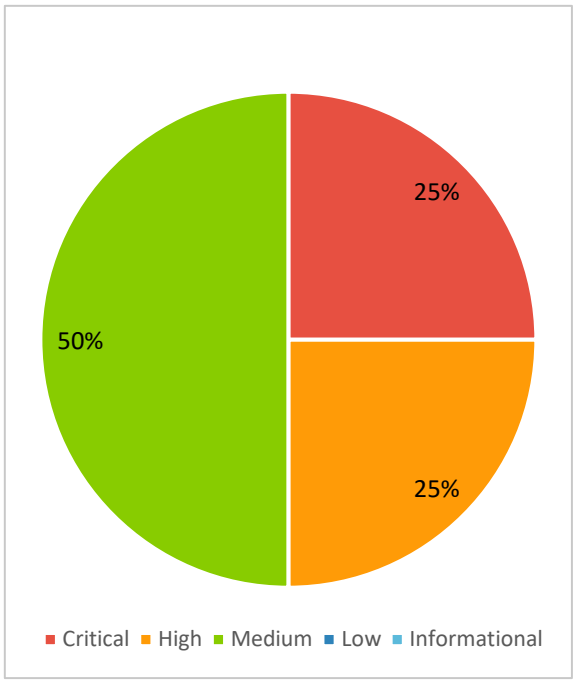


Figure 1 - Vulnerabilities By Risk

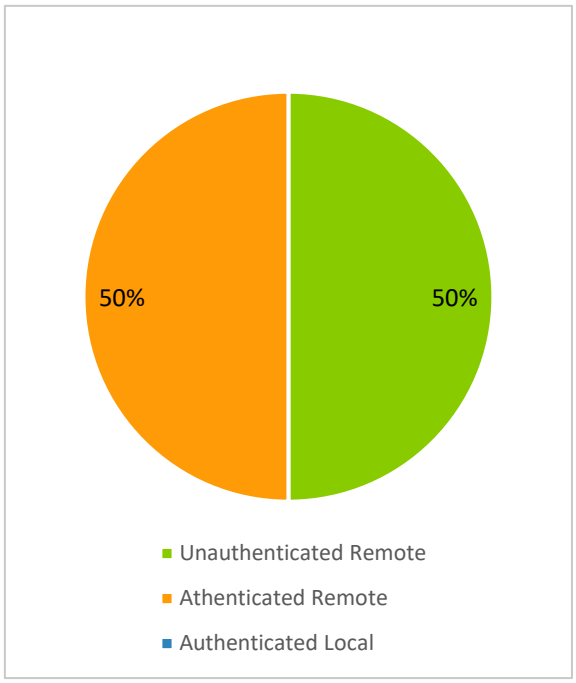


Figure 2 - Attack Perspective

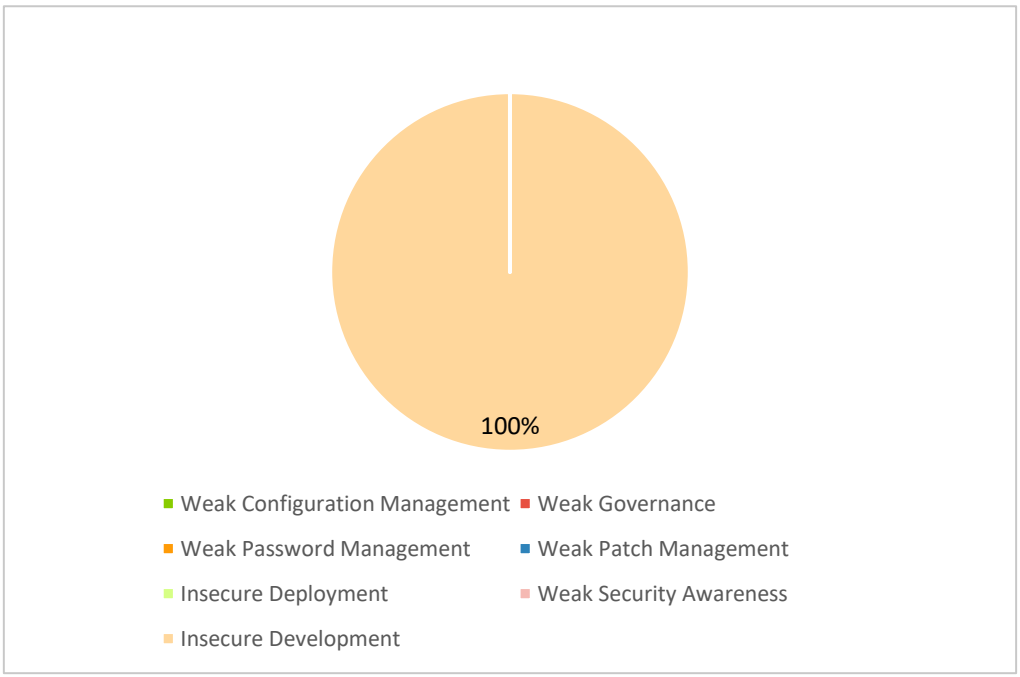


Figure 3 - Vulnerabilities VS Root Cause

3 PENETRATION TESTING CONTEXT

3.1 SCOPE AND LIMITS

This audit was limited to the DogQuest web applications, following a Grey-Box approach.

The scope of this mission is limited to the following assets:

- https://dog-quest-next-dev.vercel.app/
- https://sehzakutrlropprdcewu.supabase.co/

3.2 TESTING CONDITIONS

The objective of this engagement was to discover vulnerabilities in DogQuest web applications

The tests on the web application focused mainly on remotely exploitable vulnerabilities that could impact confidentiality, integrity, or availability.

The security tests were performed on the Development environment, following a Grey-Box approach.

The tests on the servers were performed from the machines of the security consultant.

3.3 OWASP TOP 10 - EXECUTED TEST CASES

OWASP Category	Description	Test Performed / Observation	Status
A01:2021 – Broken Access Control	Testing for access control bypass or unauthorized resource access	Access Control issues found	Observations Found
A02:2021 – Cryptographic Failures	Review of TLS configuration and data exposure in transit	No Cryptographic issues	Passed
A03:2021 – Injection	Testing for SQL, command, and code injection, Prompt injections vulnerabilities	No Injection issues	Passed
A04:2021 – Insecure Design	Assessing design flaws and potential logic weaknesses	Insecure Design Flaws Found	Observations Found
A05:2021 – Security Misconfiguration	Testing for default files, error handling, server headers, and framework disclosure	Observations found.	Observations Found
A06:2021 – Vulnerable and Outdated Components	Identifying outdated or unsupported frameworks and libraries	No vulnerable components were found.	Passed
A07:2021 – Identification and Authentication Failures	Reviewing authentication, session, and password mechanisms	No Authentication Issues were found	Passed
A08:2021 – Software and Data Integrity Failures	Reviewing CI/CD, third-party dependencies, and file integrity	No third-party misconfigurations detected	Passed
A09:2021 – Security Logging and Monitoring Failures	Verifying presence of audit/logging mechanisms and alerts	Out of scope.	Not Applicable

A10:2021 – Server-Side Request Forgery (SSRF)	Testing for SSRF via exposed endpoints or parameters	No vulnerable endpoints were detected.	Passed

4 FINDINGS AND REMEDIATIONS

4.1 SUMMARY OF RESULTS BY SERVICE/APPLICATION

Asset	Critical	High	Medium	Low	Info	Total
https://dog-quest-next-dev.vercel.app/	0	0	0	0	0	0
https://sehzakutrlropprdcewu.supabase.co	1	1	2	0	0	4

Table 7 - Summary of Results by Service/Application

5 DETAILED DESCRIPTION OF THE VULNERABILITIES

5.1 CRITICAL PRIVILEGE ESCALATION TO ADMIN VIA REGISTRATION PARAMETER MANIPULATION

Vulnerability	Critical Privilege Escalation to Admin via Registration Parameter Manipulation	**Vuln-id**	DogQuest-2026-001

Brief Description

The application fails to validate the `role` parameter during the account registration process, allowing an unauthenticated user to assign themselves administrative privileges by modifying the HTTP request.

Risk	**Critical**	**Category**	Insecure Development
Asset	https:// sehzakutrlropprdcewu.supabase.co		
CVSS Score	9.8		
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H		

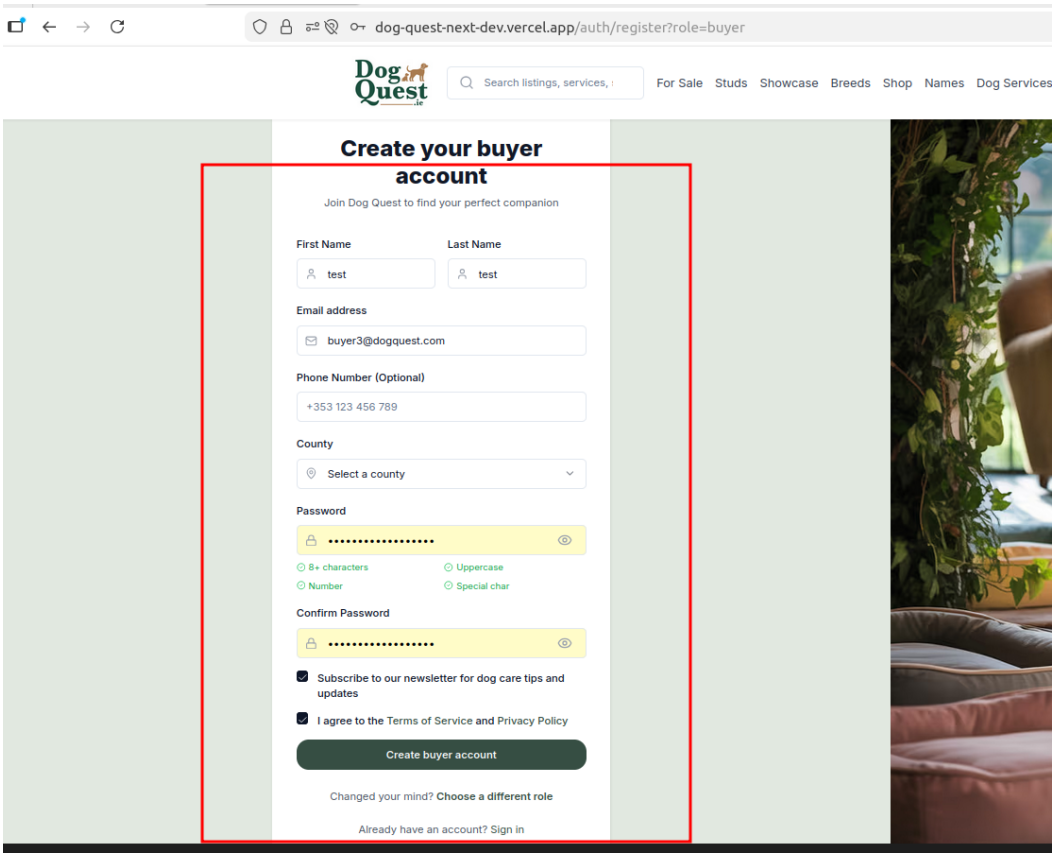
Description

The registration endpoint allows users to include a role field within the JSON body of the signup request. Because the backend (Supabase/Auth) trusts this user-supplied value without server-side verification or restriction, any user can bypass the intended "buyer" or "seller" registration workflows. By intercepting the request and changing the value to admin, an attacker can create an account with full administrative access to the platform.

Proof of Concept (PoC)

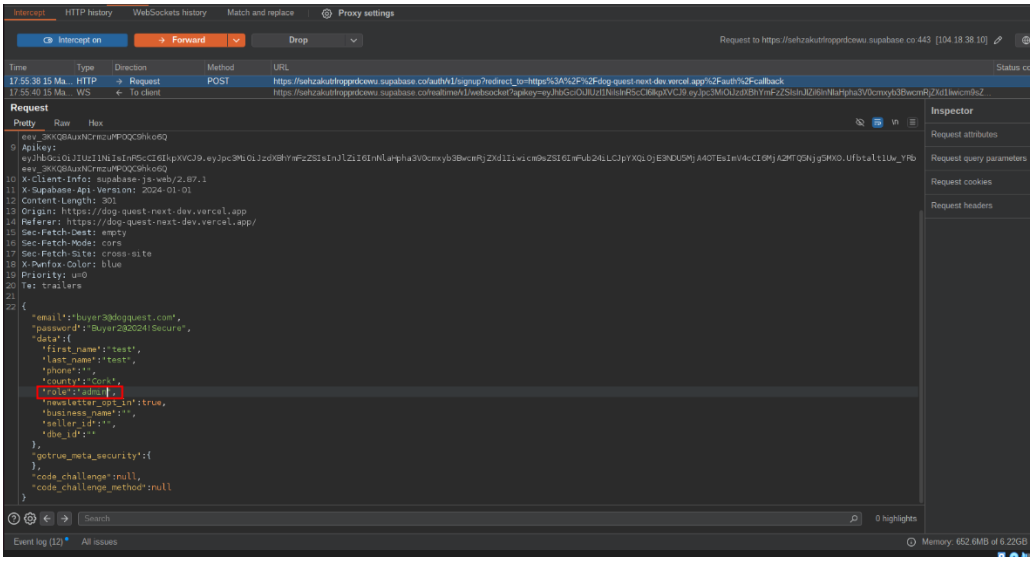
1. Navigate to the registration page: https://dog-quest-next-dev.vercel.app/auth/register?role=buyer

2. Fill in the registration form with test credentials.



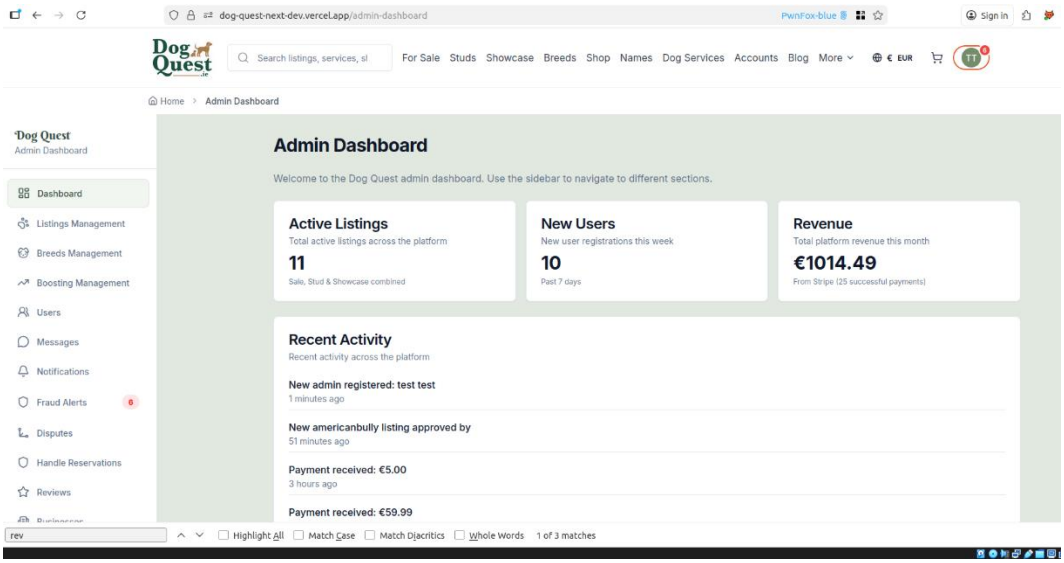
3. Intercept the outgoing POST request to the authentication provider (Supabase) using a proxy tool like Burp Suite.

4. Locate the JSON body and modify the "role": "buyer" (or similar field within data) to "role": "admin".



5. Forward the request.

6. The user is successfully registered and redirected to the /admin-dashboard, granting full access to site revenue, user management, and listing approvals.



Impact

An attacker gaining administrative access can:

- View sensitive financial data (Revenue, Stripe payment details).
- Manage, delete, or modify all user accounts and listings.
- Access internal communications and notifications.
- Potentially compromise the entire platform's integrity and user data.

Remediation

Urgency	High	**Complexity**	Low

- **Hardcode Roles:** Do not allow the role to be passed as a parameter from the client-side during public registration. The server should default all new registrations to the lowest privilege level (e.g., user or buyer).
- **Server-Side Validation:** Use a private administrative function or a protected internal API to assign administrative roles, ensuring only existing authorized admins can promote other users.
- **Role-Based Access Control (RBAC):** Implement strict Row Level Security (RLS) and policies that do not rely on a user-defined metadata field that can be set at registration.

References
CWE-213: Improper Authorization - https://cwe.mitre.org/data/definitions/285.html https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html **Authorization Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html **CWE-284: Improper Access Control** - https://cwe.mitre.org/data/definitions/284.html

5.2 GLOBAL CORS MISCONFIGURATION: ARBITRARY ORIGIN REFLECTION ACROSS ALL API ENDPOINTS

Vulnerability	Global CORS Misconfiguration: Arbitrary Origin Reflection Across All API Endpoints	**Vuln-id**	DogQuest-2026-002

Brief Description

The Supabase REST API (`/rest/v1/*`) is configured to dynamically reflect any value provided in the Origin request header into the `Access-Control-Allow-Origin` response header. This global misconfiguration allows a malicious website to bypass the Same-Origin Policy (SOP) and interact with any endpoint the victim user is authorized to access.

Risk	**High**	**Category**	Insecure Development
Asset	https:// sehzakutrlropprdcewu.supabase.co		
CVSS Score	8.1		
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N		

Description

The server at sehzakutrlropprdcewu.supabase.co fails to validate the Origin header against a whitelist of trusted domains for all REST endpoints. When a browser sends a cross-origin request, the server responds with `Access-Control-Allow-Origin: [Requesting Origin]`.

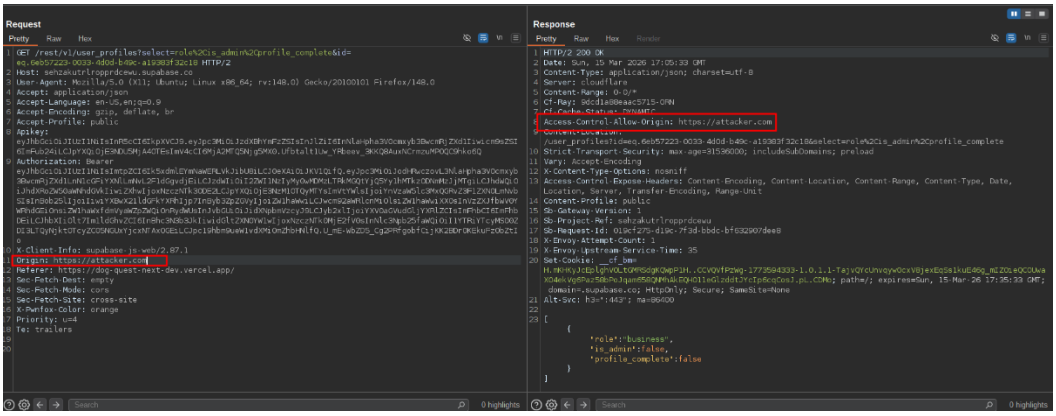
Because this behavior is global, it effectively neutralizes browser-based security for the entire API. If a victim user has an active session or a stored JWT, an attacker-controlled website can perform unauthorized actions or exfiltrate sensitive data from any table exposed via the PostgREST interface.

Proof of Concept

- **Global Testing:** Send a request to multiple endpoints (e.g., `/rest/v1/user_profiles`, `/rest/v1/orders`,

`/rest/v1/settings`) with a spoofed origin like `Origin: https://attacker.com`

2. **Response Verification:** Observe that in every instance, the server responds with the header `Access-Control-Allow-Origin: https://attacker.com` as shown in the following figure :



Impact

- **Total Data Exfiltration:** Any data readable by the victim user across the entire database can be stolen by a third-party site.
- **Unauthorized Data Modification:** Since the API allows POST and PUT requests from any origin, an attacker can create or edit records on behalf of an authenticated user, directly compromising data integrity.
- **Widespread Privacy Breach:** This affects every user of the platform, as their private data is no longer protected by the Same-Origin Policy.

Remediation

Urgency	High	**Complexity**	Low

1. **Enforce Strict Origin Whitelisting:** Update the server configuration to only allow specific, trusted origins (e.g., your production and staging URLs).
2. **Disable Dynamic Reflection:** The server should never echo the Origin header directly without verification.

3. **Supabase-Specific Fix:** In the Supabase dashboard, navigate to **Project Settings > API** and ensure that the "CORS Allowed Origins" field contains only your specific domains, rather than a wildcard or a configuration that reflects the request origin.

References

CWE-942: Permissive Cross-Domain Policy with Untrusted Domains - https://cwe.mitre.org/data/definitions/942.html

OWASP Custom Headers and CORS: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html#custom-headers-and-cors

5.3 IMPROPER ACCESS CONTROL AND MASS DISCLOSURE OF USER PROFILES VIA /REST/V1/USER_PROFILES

Vulnerability	Improper Access Control and Mass Disclosure of User Profiles via /rest/v1/user_profiles	**Vuln-id**	DogQuest-2026-003
Brief Description			
An authenticated attacker can bypass intended single-row response constraints to retrieve the entire user_profiles database table. This leads to the exposure of sensitive PII (Personally Identifiable Information) and system metadata for all registered users.			

Risk	**Medium**	**Category**	Insecure Development
Asset	https:// sehzakutrlropprdcewu.supabase.co		
CVSS Score	6.5		
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N		
Description			

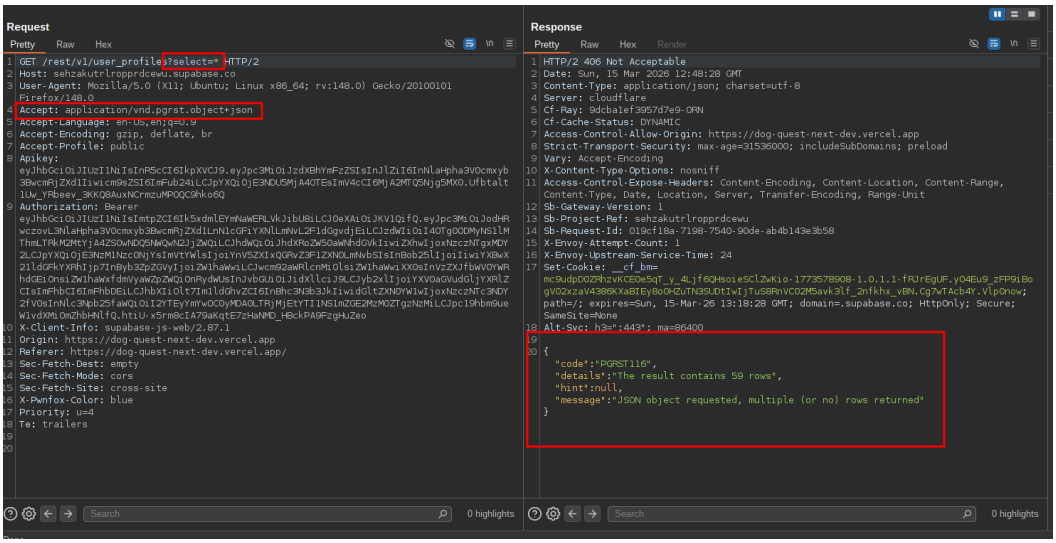
The application utilizes a Supabase (PostgREST) backend. By default, the frontend sends a request to `/rest/v1/user_profiles` with the header `Accept: application/vnd.pgrst.object+json`. This header instructs the server to return a single JSON object; if multiple rows are found, the server returns a 406 Not Acceptable error (Error Code PGRST116), as seen in the provided Proof of Concept.

However, the backend lacks **Row Level Security (RLS)** or proper server-side filtering. An attacker can remove this specific header or modify it to `application/json`, causing the server to return a full JSON array containing every record in the table. The leaked data includes:

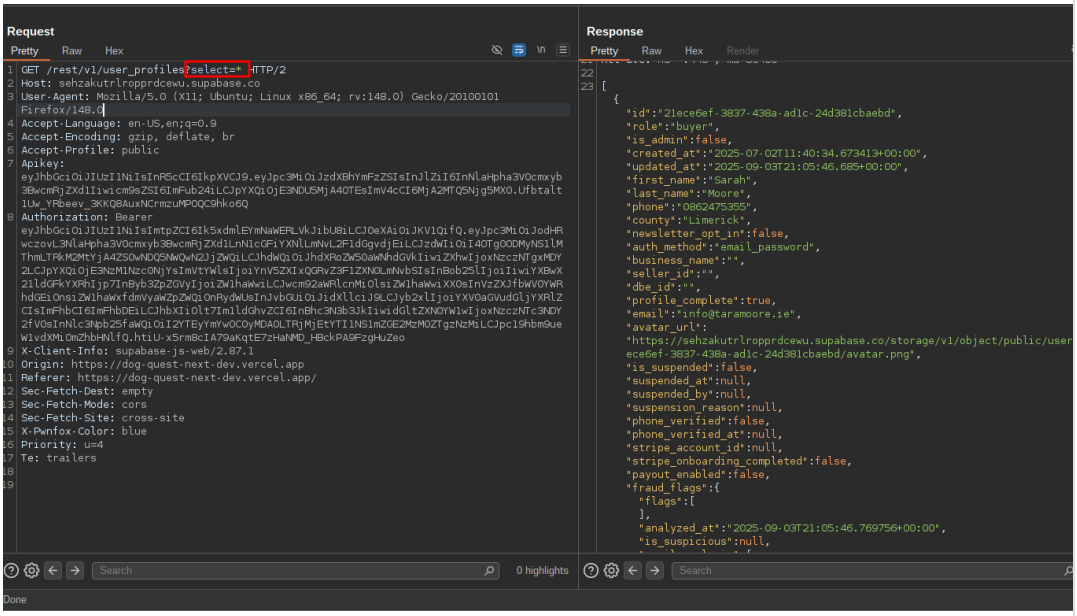
- Full names and email addresses.
- Phone numbers and location data (County).
- Unique User IDs (UUIDs).
- Internal flags such as is_admin, is_suspended, and is_suspicious.

Proof of Concept

1. **Authentication:** Authenticate as a standard, low-privileged user.
2. **Request Manipulation:** Intercept the request to: `GET /rest/v1/user_profiles?select=*`

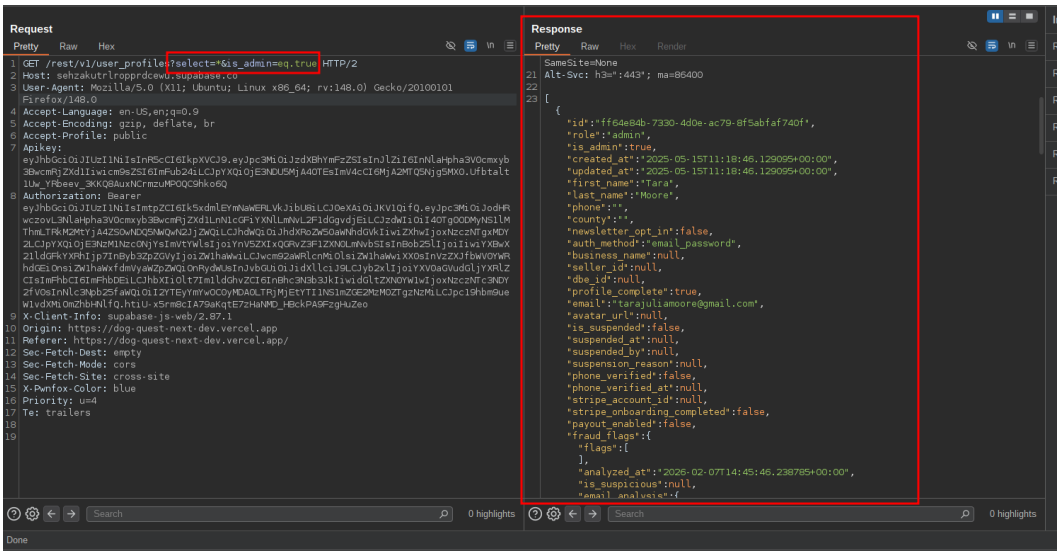


3. **Bypass:** Remove the header Accept: `application/vnd.pgrst.object+json`.
4. **Mass Extraction:** The server responds with a 200 OK and a full list of all 59+ users in the database (as shown in the following capture).



5. **Targeted Filtering:** The attacker can further refine the leak to identify high-value targets by appending query parameters, such as: `GET`

`/rest/v1/user_profiles?select=*&is_admin=eq.true` (as shown in the following capture).



Impact

- **High Confidentiality Loss:** Total exposure of the user database.
- **Privilege Escalation Vector:** By identifying users with the `is_admin: true flag`, attackers can focus exploitation efforts on high-privilege accounts.
- **Privacy Compliance:** Violation of data protection regulations (e.g., GDPR, CCPA) due to the exposure of emails and phone numbers.

Remediation

Urgency	High	Complexity	Low

- **Enable Row Level Security (RLS):** Ensure RLS is enabled for the user_profiles table in Supabase.
- **Implement Access Policies:** Define a PostgreSQL policy that restricts users to only viewing their own data.

```
SQL
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own profile"
ON user_profiles FOR SELECT
USING (auth.uid() = id);
```

- **Restrict Sensitive Columns:** Even if some profile data must be public, sensitive columns (like `is_admin`, `is_suspended`, phone) should be moved to a separate table or restricted via a View that does not expose them to the standard API role.

References

- **Supabase Row Level Security**: https://supabase.com/docs/guides/database/postgres/row-level-security
- **Authorization Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- **CWE-284: Improper Access Control -** https://cwe.mitre.org/data/definitions/284.html

5.4 USER AUTHENTICATION METHOD ENUMERATION VIA /REST/V1/RPC/GET_AUTH_METHOD_BY_EMAIL

Vulnerability	User Authentication Method Enumeration via /rest/v1/rpc/get_auth_method_by_email	**Vuln-id**	DogQuest-2026-004
Brief Description			
An authenticated attacker can determine the specific login method (e.g., email_password, google_oauth, etc.) for any user by querying their email address. This disclosure assists in targeted credential-based attacks.			

Risk	**Medium**	**Category**	Insecure Development
Asset	https:// sehzakutrlropprdcewu.supabase.co		
CVSS Score	5.3		
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N		
Description			

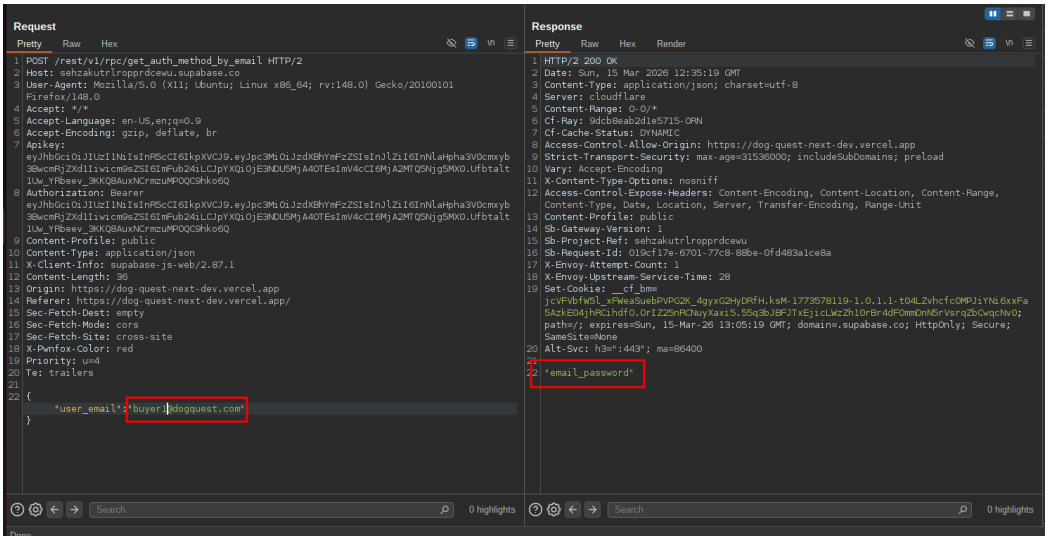
The application exposes a Remote Procedure Call (RPC) endpoint, `/rest/v1/rpc/get_auth_method_by_email`, which is accessible to any unauthenticated user. By providing a target user's email address in the request body, the server returns the authentication provider associated with that account.

This information is sensitive metadata. Knowing whether a user relies on a password or a third-party SSO provider allows an attacker to tailor their exploitation strategy, focusing brute-force efforts only on password-based accounts, or crafting specific phishing pages for SSO users.

Proof of Concept

1. **Request Execution:** Send a POST request to the RPC endpoint.

2. **Response Observation:** The server returns a 200 OK with the auth method.



Impact

- **Targeted Reconnaissance:** Attackers can filter large lists of leaked emails to identify those most vulnerable to password-based attacks.
- **Advanced Phishing:** Attackers can create highly convincing, provider-specific phishing pages (e.g., a fake Google login if the method is google_oauth).

Remediation

Urgency	High	Complexity	Low

- **Enforce Authentication:** Change the function's permission level in PostgreSQL/Supabase so it is only executable by the service_role or a specific admin role.
- **Remove Public Access:** In the Supabase Dashboard, ensure the function is not exposed to the anon or public roles.
- **Rate Limiting:** If the endpoint must remain public for architectural reasons, implement aggressive IP-based rate limiting to prevent bulk enumeration.

References

CWE-200: Exposure of Sensitive Information to an Unauthorized Actor - https://cwe.mitre.org/data/definitions/200.html